

TypeScript面试题(50题)

1. 说说你对 TypeScript 中命名空间与模块的理解？区别？

参考答案:

- **模块 (Modules) :**

TypeScript 与 ECMAScript 2015 一样，任何包含顶级 `import` 或者 `export` 的文件都被当成一个模块。模块有自己的作用域，模块中定义的变量、函数、类等在外部是不可见的，除非使用 `export` 导出。相反，如果一个文件不带有顶级的 `import` 或者 `export` 声明，那么它的内容被视为全局可见的。模块是组织代码的首选方式，尤其是在大型应用中。

- **命名空间 (Namespaces) :**

命名空间是 TypeScript 早期的模块化方案，现在主要用于组织全局变量，避免命名冲突。它通过 `namespace` 关键字定义，可以包含变量、函数、类和接口等。命名空间可以跨多个文件，并且可以使用 `/// <reference path="..." />` 指令来引用。在现代 TypeScript 开发中，推荐使用 ES 模块替代命名空间。

- **区别:**

- **作用域:** 模块是文件级别的作用域，而命名空间是全局作用域下的一个对象。
- **依赖管理:** 模块通过 `import` / `export` 显式声明依赖，而命名空间通常需要通过 `<reference>` 标签或打包工具来管理文件顺序。
- **生态系统:** ES 模块是 JavaScript 的标准，拥有更广泛的社区支持和工具链兼容性。
- **推荐使用:** 在新项目中，应优先使用模块。命名空间主要用于维护旧项目或在特殊场景下组织全局变量。

2. `type` 和 `interface` 有什么区别？

参考答案:

`type` (类型别名) 和 `interface` (接口) 都可以用来定义对象或函数的类型，但在功能上存在一些差异。

- 相同点:

- 都可以描述对象的形状或函数签名。
- 都支持继承，`interface` 使用 `extends`，`type` 使用交叉类型 `&`。

- 不同点:

- **扩展性:** `interface` 可以重复声明并会自动合并 (Declaration Merging)，这使得扩展第三方库的接口变得容易。而 `type` 不支持重复声明。
- **实现:** 类可以 `implements` 一个 `interface` 或 `type`，但 `interface` 只能 `extends` 另一个 `interface`。
- **原始类型:** `type` 可以为原始类型 (如 `string`，`number`)、联合类型、元组等创建别名，而 `interface` 主要用于描述对象的结构。
- **计算属性:** `type` 可以使用 `typeof`、`keyof` 等操作符创建更复杂的类型，而 `interface` 不支持。

- 选择建议:

- 当定义公共 API 或希望对象结构可被扩展时，优先使用 `interface`。
- 当需要定义联合类型、元组或处理复杂的类型运算时，使用 `type`。

3. TypeScript 中的 `any` 和 `unknown` 有什么区别?

参考答案:

`any` 和 `unknown` 都代表任意类型，但 `unknown` 是 `any` 的类型安全版本。

- `any` :
 - 表示任意类型，TypeScript 编译器会对其完全放弃类型检查。
 - 可以对 `any` 类型的变量进行任何操作，包括属性访问、函数调用等，这在编译时都不会报错，但可能在运行时出错。
 - `any` 类型的变量可以赋值给任何其他类型的变量。
 - `any` 会在类型系统中“传播”，任何与之交互的值都会“被污染”成 `any` 类型。
 - `unknown` :
 - 表示一个未知的类型，是所有类型的父类型。
 - 不能对 `unknown` 类型的变量进行任何操作（除了赋值给 `any` 或 `unknown` 类型），除非通过类型断言、类型收窄（如 `typeof`、`instanceof` 判断）来明确其具体类型。
 - `unknown` 类型的变量只能赋值给 `any` 或 `unknown` 类型的变量。
 - **总结:** `unknown` 强制开发者在执行操作前进行类型检查，从而保证了代码的类型安全，应在不确定类型时优先使用 `unknown` 而非 `any`。
-

4. 什么是 TypeScript 中的泛型（Generics）？它有什么作用？

参考答案:

泛型（Generics）是一种在定义函数、类或接口时不预先指定具体类型，而在使用时再指定类型的一种特性。它允许我们编写可重用的、类型安全的代码。

- 作用:

- a. **代码重用**: 泛型允许我们编写一个可以处理多种数据类型的组件（函数、类等），而不需要为每种类型都重写一遍代码。例如，一个 `identity` 函数可以接收任何类型的参数并返回相同类型的值。
- b. **类型安全**: 泛型在提供灵活性的同时，保持了严格的类型检查。它能在编译时捕获类型错误，而不是在运行时。例如，一个泛型集合类，在创建实例时指定其元素类型，之后添加或获取元素时，编译器都会检查类型是否匹配。
- c. **抽象和封装**: 泛型可以帮助我们创建更通用的抽象，隐藏具体实现的类型细节。

- 示例:

代码块

```
1  function identity<T>(arg: T): T {  
2      return arg;  
3  }  
4  
5  let output = identity<string>("myString"); // 类型为 string  
6  let output2 = identity(100); // 类型被推断为 number
```

5. `never` 类型在 TypeScript 中有什么用？

参考答案:

`never` 类型表示的是那些永不存在的值的类型。它主要用于以下场景：

1. **总是抛出异常的函数**: 如果一个函数总是抛出异常，那么它永远不会有返回值，其返回类型就是 `never`。

```
1 function error(message: string): never {
2     throw new Error(message);
3 }
```

2. **无限循环的函数:** 如果一个函数中存在无法结束的循环（如 `while(true) {}`），它也永远不会返回，其返回类型也是 `never`。
3. **类型收窄的完备性检查:** 在 `switch` 或 `if/else` 语句中，我们可以利用 `never` 类型的特性来确保我们已经处理了所有可能的情况。如果所有情况都被覆盖，`never` 类型可以被赋值；如果遗漏了某个情况，编译器会报错，因为一个具体的类型无法赋值给 `never`。

代码块

```
1 type Shape = "square" | "circle";
2
3 function getArea(shape: Shape) {
4     switch (shape) {
5         case "square": return 1;
6         case "circle": return Math.PI;
7         default:
8             const _exhaustiveCheck: never = shape; // 如果有新的 Shape 类型未
              处理，这里会报错
9             return _exhaustiveCheck;
10    }
11 }
```

6. 什么是 TypeScript 的类型断言（Type Assertion）？

参考答案:

类型断言是一种向编译器提供有关我们比它更了解某个值的类型信息的方式。它类似于其他语言中的类型转换，但它不会进行任何运行时的特殊检查或数据重构。它只在编译阶段起作用，并且假设开发者已经进行了必要的检查。

- 语法:

- a. 尖括号语法: `<Type>value`

- b. `as` 语法**: `value as Type` (在 JSX/TSX 中, 只能使用 `as` 语法, 以避免与 JSX 标签混淆, 因此 `as` 是更推荐的写法)。

- 示例:

代码块

```
1 let someValue: unknown = "this is a string";
2 let strLength: number = (someValue as string).length;
```

- **注意:** 类型断言不应滥用。如果可以, 应优先使用类型守卫 (Type Guards) 等更安全的类型收窄方法。断言错误可能导致运行时错误。

7. 解释一下 TypeScript 中的 `declare` 关键字。

参考答案:

`declare` 关键字用于向 TypeScript 编译器“声明”一个在其他地方 (如另一个 JavaScript 文件、浏览器环境或 Node.js 环境) 已经存在的变量、函数、类或模块。

它的主要作用是让 TypeScript 知道这些实体的存在及其类型, 从而在编译时能够进行类型检查, 而不会因为找不到定义而报错。 `declare` 只包含类型定义, 不包含具体的实现。

- 常见用途:

- a. 声明全局变量: `declare const $: any;` (声明一个全局的 jQuery 对象)

- b. **声明模块**: `declare module 'some-js-library';` (为没有 TypeScript 类型定义的 JavaScript 库提供一个基本的模块声明)
 - c. **声明文件** (`.d.ts`)**: 在声明文件中, `declare` 被广泛用于描述现有 JavaScript 代码的类型结构。
-

8. 什么是 TypeScript 的装饰器 (Decorators) ? 它有什么应用场景?

参考答案:

装饰器是一种特殊的声明, 它可以附加到类声明、方法、访问器、属性或参数上。装饰器使用 `@expression` 的形式, `expression` 求值后必须为一个函数, 它会在运行时被调用, 被装饰的声明信息作为其参数。

装饰器是 ES7 的一个提案, TypeScript 较早地提供了实验性支持。

• 应用场景:

- a. **AOP (面向切面编程)**: 在不修改原有代码的情况下, 为类或方法添加额外的行为, 如日志记录、性能监控、事务处理等。
 - b. **元数据编程**: 配合 `reflect-metadata` 库, 可以为类和属性附加元数据, 用于依赖注入 (DI) 容器 (如 Angular, NestJS)、ORM (如 TypeORM) 等。
 - c. **代码转换/增强**: 自动绑定 `this`、定义 Web 框架的路由、验证模型属性等。
-
- **注意**: 要使用装饰器, 需要在 `tsconfig.json` 中开启 `experimentalDecorators` 和 `emitDecoratorMetadata` 选项。
-

9. TypeScript 中的 `enum` (枚举) 是什么? 它和 `const enum` 的区别?

参考答案:

`enum`（枚举）是 TypeScript 中用于定义一组命名常量的数据结构。它有助于提高代码的可读性和可维护性，通过名字而不是魔术数字来表示一组固定的值。

- `enum` :

- 默认情况下，枚举是基于数字的，第一个成员默认为 `0`，其余成员依次递增。也可以手动设置成员的值。
- 枚举成员也可以是字符串。
- TypeScript 会为数字枚举生成一个反向映射（Reverse Mapping），即可以从枚举名得到值，也可以从值得到枚举名。
- 编译后会生成一个真实存在的 JavaScript 对象。

- `const enum` :

- 常量枚举使用 `const` 关键字修饰。
- 它在编译后会被完全移除，所有使用到枚举成员的地方都会被直接替换为对应的内联值。
- 它不能有计算成员。
- 这样做的好处是可以减少编译后的代码体积，提升性能。

- 区别总结:

- **编译结果:** `enum` 编译成一个 JavaScript 对象，`const enum` 则被完全抹除，成员被内联。
- **反向映射:** `const enum` 不支持反向映射。
- **性能:** `const enum` 的性能更好，因为它避免了额外的对象查找和属性访问。

10. `keyof` 和 `typeof` 在 TypeScript 中有什么作用？

参考答案:

`keyof` 和 `typeof` 是 TypeScript 中用于类型操作的两个重要操作符。

- `keyof` (索引类型查询操作符)**:

- 它接收一个对象类型，并返回该类型所有公共属性名组成的联合类型（string literal union type）。
- 示例:

代码块

```
1 interface Person {  
2     name: string;  
3     age: number;  
4 }  
5 type PersonKeys = keyof Person; // "name" | "age"
```

- `typeof` (类型查询操作符)**:

- 在类型上下文中（即 `type` 别名或泛型约束等地方），`typeof` 用于获取一个变量或属性的类型。
- 它与 JavaScript 中的 `typeof` 运算符不同，JavaScript 的 `typeof` 在运行时返回值的字符串表示（如 "string", "number"），而 TypeScript 的 `typeof` 在编译时获取类型信息。
- 示例:

代码块

```
1 const person = { name: "Alice", age: 30 };  
2 type PersonType = typeof person; // { name: string; age: number; }
```

11. 什么是 TypeScript 中的类型守卫（Type Guards）？

参考答案: 类型守卫是在运行时执行的表达式，用于确保一个变量在某个作用域内是特定的类型。常见的类型守卫有 `typeof`，`instanceof`，`in` 操作符，以及自定义的类型谓词函数（`x is Type`）。

12. 解释一下 TypeScript 的 `infer` 关键字。

参考答案: `infer` 关键字出现在条件类型（`extends`）的 `true` 分支中，用于声明一个类型变量，并从正在匹配的类型中推断出这个变量的类型。它常用于获取函数参数类型、返回值类型、Promise 的 `resolve` 类型等。

13. 什么是映射类型（Mapped Types）？

参考答案: 映射类型是一种泛型类型，它使用一个已知的类型，并根据其属性来创建一个新的类型。它通过 `in keyof` 语法遍历一个类型的键，并为新类型的每个键定义类型。常用于创建只读、可选或部分类型的变体。

14. 什么是条件类型（Conditional Types）？

参考答案: 条件类型是 TypeScript 中一种根据条件选择两种类型之一的类型。它的形式是 `T extends U ? X : Y`。如果 `T` 可以赋值给 `U`，则结果是 `X` 类型，否则是 `Y` 类型。它使得类型可以进行类似三元运算符的逻辑判断。

15. `public`，`private`，`protected` 修饰符有什么区别？

参考答案:

- `public`：默认修饰符，类的成员在任何地方都可以被访问。
- `private`：成员只能在声明它的类内部访问。
- `protected`：成员可以在声明它的类及其子类中访问。

16. TypeScript 中的 `abstract class`（抽象类）是什么？

参考答案: 抽象类是作为其他派生类的基类的类。它不能被直接实例化。抽象类可以包含抽象方法（只有方法签名，没有实现），派生类必须实现这些抽象方法。

17. `tsconfig.json` 文件有什么作用？

参考答案: `tsconfig.json` 文件是 TypeScript 项目的根文件。它指定了编译项目所需的根文件和编译器选项。通过这个文件，我们可以配置目标 JavaScript 版本、模块系统、是否启用严格模式、源映射等。

18. 什么是声明文件 (`.d.ts`)？

参考答案: 声明文件 (`.d.ts`) 是用来为 JavaScript 库或模块提供类型信息的。它只包含类型声明，不包含具体实现。这使得我们可以在 TypeScript 项目中安全地使用纯 JavaScript 编写的库，并获得代码提示和类型检查。

19. 什么是 TypeScript 的结构化类型系统 (Structural Typing) ？

参考答案: TypeScript 的类型系统是结构化的（也称为“鸭子类型”）。这意味着如果两个类型具有相同的结构（属性和方法），那么它们就是兼容的，而不管它们的名称是什么。只要一个对象满足了某个接口的结构要求，它就被认为是该接口的类型。

20. `T | null` 和 `T & null` 分别是什么意思？

参考答案:

- `T | null`: 联合类型 (Union Type)。表示一个值可以是 `T` 类型，也可以是 `null`。
- `T & null`: 交叉类型 (Intersection Type)。通常结果是 `null`，因为一个类型不能同时是 `T` 和 `null`（除非 `T` 是 `any` 或 `unknown`）。交叉类型用于合并多个类型的成员。

21. 解释 TypeScript 中的 `this` 和 `=>` (箭头函数)。

参考答案: 和 JavaScript 一样，普通函数中的 `this` 是动态绑定的，取决于函数的调用方式。而箭头函数 (`=>`) 中的 `this` 是词法绑定的，它会捕获其所在上下文的 `this` 值，这在类的方法或回调函数中非常有用，可以避免 `this` 指向混乱的问题。

22. 什么是元组 (Tuple) 类型？

参考答案: 元组类型允许你表示一个已知元素数量和类型的数组。元组中每个元素的位置都有固定的类型。例如 `[string, number]` 表示一个数组，其第一个元素是字符串，第二个元素是数字。

23. TypeScript 是如何进行类型推断的？

参考答案: 当我们没有显式指定类型时，TypeScript 编译器会根据上下文自动推断出变量、函数返回值等的类型。例如，`let x = 10;` TypeScript 会推断 `x` 的类型为 `number`。它也会根据函数体的 `return` 语句推断函数的返回类型。

24. `void` 类型有什么用？

参考答案: `void` 类型表示没有任何类型。它通常用作不返回任何值的函数的返回类型。`void` 类型的变量只能被赋值为 `undefined` 或 `null`（在非严格空检查模式下）。

25. `satisfies` 操作符是做什么的？

参考答案: `satisfies` 操作符（TypeScript 4.9+）用于在不改变表达式原有类型推断的情况下，验证该表达式的类型是否满足某个更宽泛的类型。这有助于在保持具体类型的同时，确保其符合某个接口或规范，从而在后续使用时获得更精确的类型提示。

26. 什么是索引签名（Index Signatures）？

参考答案: 索引签名用于描述那些“通过索引访问”的对象的类型，比如具有动态属性名的对象。语法是 `[key: T]: U`，其中 `T` 必须是 `string` 或 `number`，`U` 是值的类型。

27. TypeScript 的严格空检查 (`strictNullChecks`) 是什么？

参考答案: 这是一个编译器选项。当它被启用时，`null` 和 `undefined` 值将不能赋值给其他类型的变量，除非显式地使用联合类型（如 `string | null`）。这可以帮助开发者在编译阶段就发现潜在的 `null` 或 `undefined` 错误。

28. 如何在 TypeScript 中使用 `async/await`？

参考答案: TypeScript 完全支持 `async/await`。一个 `async` 函数的返回值总是被包装在一个 `Promise` 中。`await` 关键字只能在 `async` 函数内部使用，用于等待一个 `Promise` 解析并返回其结果。TypeScript 会对 `Promise` 的 `resolve` 值进行类型推断和检查。

29. `export default` 和 `export` 的区别是什么？

参考答案:

- `export`: 可以导出多个命名成员（变量、函数、类）。导入时需要使用花括号 `{}` 并且名称必须匹配，例如 `import { a, b } from './module'`。
- `export default`: 每个模块只能有一个默认导出。导入时不需要花括号，并且可以任意命名，例如 `import myModule from './module'`。

30. 什么是“工具类型”（Utility Types）？请举例。

参考答案: 工具类型是 TypeScript 内置的一些泛型类型，用于帮助我们进行常见的类型转换。

- `Partial<T>`: 将 `T` 的所有属性变为可选。
- `Readonly<T>`: 将 `T` 的所有属性变为只读。
- `Pick<T, K>`: 从 `T` 中选择一组属性 `K` 来构造一个新的类型。
- `Omit<T, K>`: 从 `T` 中移除一组属性 `K` 来构造一个新的类型。
- `Record<K, T>`: 构造一个对象类型，其属性键为 `K`，属性值为 `T`。

31. TypeScript 源码是如何编译成 JavaScript 的？

参考答案: TypeScript 编译器 (TSC) 会解析 `.ts` 文件，进行类型检查，然后将 TypeScript 语法（如类型注解、接口、泛型等）擦除，并根据 `tsconfig.json` 中的 `target` 选项将 ESNext 语法（如类、箭头函数）转换为目标版本的 JavaScript 代码。

32. `readonly` 修饰符有什么作用？

参考答案: `readonly` 修饰符用于将类的属性或接口的属性标记为只读。一旦一个只读属性在声明时或构造函数中被初始化后，就不能再被修改。这有助于创建不可变的数据结构。

33. 什么是类型别名（Type Aliases）？

参考答案: 类型别名使用 `type` 关键字为一个类型起一个新的名字。它可以用于原始类型、联合类型、交叉类型、元组，以及任何其他你需要命名的类型。它不会创建新的类型，只是创建一个引用。

34. TypeScript 如何处理函数重载？

参考答案: 在 TypeScript 中，可以为同一个函数提供多个函数类型定义（重载签名），然后紧跟一个通用的实现签名。编译器在调用函数时，会根据传入的参数类型从上到下匹配最合适的重载签名进行类型检查。

35. `is` 关键字的作用是什么？

参考答案: `is` 关键字用于创建用户自定义的类型守卫函数，也称为类型谓词。当一个函数返回 `arg is Type` 时，如果函数返回 `true`，TypeScript 编译器就会在后续的代码块中将 `arg` 的类型收窄为 `Type`。

36. 如何理解协变（Covariance）和逆变（Contravariance）？

参考答案:

- **协变:** 如果 `Dog` 是 `Animal` 的子类型，那么 `Array<Dog>` 也是 `Array<Animal>` 的子类型。这在对象属性、数组和函数返回值类型中是安全的。
- **逆变:** 如果 `Dog` 是 `Animal` 的子类型，那么 `(dog: Dog) => void` 却是 `(animal: Animal) => void` 的父类型。这主要体现在函数参数类型上，参数类型允许更宽泛的类型。

37. TypeScript 中的 Mixins 是什么？如何实现？

参考答案: Mixins 是一种代码复用模式，它允许一个类从多个辅助类（Mixin 类）中“混入”属性和方法。在 TypeScript 中，可以通过类表达式、接口合并和 `Object.assign` 等技术来实现，将多个类的功能组合到一个类中。

38. `unknown` 和 `any` 在 JSON 解析场景下如何选择？

参考答案: 应该选择 `unknown`。因为从外部（如 API）获取的 JSON 数据结构是不确定的，使用 `unknown` 会强制开发者在使用数据前进行安全的类型检查（如属性存在性检查、类型断言），而 `any` 则会绕过所有类型检查，容易引入运行时错误。

39. 什么是字面量类型（Literal Types）？

参考答案: 字面量类型允许你将变量的类型限制为某个具体的字面量值。它可以是字符串字面量、数字字面量或布尔字面量。它常与联合类型结合使用，来限制变量只能取几个特定的值之一。

40. 如何优雅地处理可选链 (?.) 和空值合并 (??)?

参考答案:

- **可选链 (?.)**:** 用于安全地访问深层嵌套对象的属性。如果链中的任何一个属性是 `null` 或 `undefined`，整个表达式会短路并返回 `undefined`，避免了 `TypeError`。
- **空值合并 (??)**:** 当左侧操作数为 `null` 或 `undefined` 时，返回右侧的操作数，否则返回左侧的操作数。它与 `||` 的区别在于 `||` 会对所有 "falsy" 值（如 `0`，`''`，`false`）生效。

41. 什么是模板字面量类型 (Template Literal Types) ?

参考答案: 模板字面量类型 (TypeScript 4.1+) 允许我们基于字符串字面量类型创建新的字符串字面量类型。它使用与模板字符串相同的语法，但用在类型位置。它可以用于拼接、模式匹配和转换字符串类型。

42. Awaited<T> 工具类型是做什么的?

参考答案: `Awaited<T>` 工具类型用于获取一个 `Promise` 的 `resolve` 值的类型，它可以递归地“解包”嵌套的 `Promise`。例如 `Awaited<Promise<Promise<string>>>` 的结果是 `string` 类型。

43. TypeScript 支持 CommonJS 和 ES Modules 吗? 如何配置?

参考答案: 支持。可以在 `tsconfig.json` 的 `compilerOptions` 中通过 `module` 字段来配置。设置为 `"commonjs"` 会编译成 CommonJS 模块（使用 `require` / `module.exports`），设置为 `"esnext"` 或 `"es2020"` 等会编译成 ES 模块（使用 `import` / `export`）。

44. 解释一下 TypeScript 的“名义化类型”模拟。

参考答案: TypeScript 是结构化类型系统，有时我们需要名义化类型（即类型名称不同就视为不同类型）。可以通过品牌化 (Branding) 技术来模拟，即在类型中添加一个唯一的、私有的品牌属性（如 `__brand: 'UserId'`），使得即使两个类型结构相同，由于品牌不同，它们也不再兼容。

45. 什么是 .tsbuildinfo 文件?

参考答案: `.tsbuildinfo` 文件是 TypeScript 在启用增量编译（`--incremental` 或 `composite: true`）时生成的文件。它存储了上次编译的构建信息，使得下一次编译时，编译器

可以只重新编译发生变化的文件及其依赖，从而大大加快编译速度。

46. `infer` 与泛型参数有什么不同？

参考答案: 泛型参数 `<T>` 是由调用者在使用时传入的具体类型。而 `infer R` 是在条件类型内部，由编译器根据上下文自动推断出来的类型。前者是“输入”，后者是“输出”或“推断结果”。

47. 如何在 TypeScript 项目中引入一个没有类型定义的 npm 包？

参考答案:

1. 首先尝试在 `@types` 组织下安装它的声明文件，例如 `npm install @types/package-name`。
2. 如果不存在，可以在项目中创建一个全局的 `.d.ts` 文件（如 `global.d.ts`），然后使用 `declare module 'package-name';` 来声明这个模块，这样 TypeScript 至少不会报错，但会将其类型视为 `any`。
3. 为了更好的类型安全，可以在声明模块内部为其关键的 API 添加类型定义。

48. 什么是 Variance（变体）？它对函数类型有什么影响？

参考答案: Variance 描述了类型构造器如何处理子类型关系，主要有协变、逆变和双变。对于函数类型 `(arg: A) => B`，它的参数类型 `A` 是逆变的（允许更通用的类型），而返回值类型 `B` 是协变的（允许更具体的类型）。默认情况下，TypeScript 的函数参数是双变的（Bivariant），但可以通过开启 `strictFunctionTypes` 选项使其变为更安全的逆变。

49. `Omit<T, K>` 和 `Exclude<T, U>` 有什么区别？

参考答案:

- `Omit<T, K>`: 用于**对象类型**。它从对象类型 `T` 中移除指定的属性键 `K`，返回一个新的对象类型。
- `Exclude<T, U>`: 用于**联合类型**。它从联合类型 `T` 中排除所有可以赋值给 `U` 的类型成员。

50. 你认为 TypeScript 最大的优势和潜在的缺点是什么？

参考答案:

- **优势:**

- **类型安全:** 在编译阶段发现大量潜在错误，提高代码质量和可维护性。
- **代码可读性和可维护性:** 类型注解使得代码意图更清晰，易于理解和重构。
- **强大的工具支持:** 提供了优秀的自动补全、代码导航和重构功能。
- **渐进式引入:** 可以与现有 JavaScript 项目无缝集成，逐步迁移。
- **面向对象编程:** 提供了类、接口、泛型等完整的面向对象特性。

- **潜在缺点:**

- **学习曲线:** 需要学习额外的类型系统概念。
- **开发成本:** 需要编写类型定义，增加了前期的开发工作量。
- **编译步骤:** 需要一个编译步骤才能运行，可能使构建过程复杂化。
- **类型体操:** 过于复杂的类型操作有时会降低代码的可读性。